

Task Board Application

Uses, Advantages (Pros) & Limitations (Cons) Report

GitHub Repository: <https://github.com/Ragul-j-1/daily-with-code>

1. Executive Summary

This evaluation document provides an objective analysis of the **Task Board** web application. The project source code is hosted on GitHub at <https://github.com/Ragul-j-1/daily-with-code>. This report outlines the core practical uses of the system, details its key technical and user experience advantages (Pros), and identifies current limitations (Cons) alongside recommendations for future enhancement.

2. Primary Uses & Use Cases

- **Personal Task Management:** Helps individuals organize, track, and manage daily to-do lists, project milestones, and personal errands efficiently.
- **Priority-Based Workflow Organization:** Allows users to categorize tasks into High, Medium, and Low priorities to focus on urgent items first.
- **Deadline & Due Date Tracking:** Enables tracking target completion dates with instant visual status alerts (e.g. *Overdue*, *Due Today*, *Upcoming*).
- **Productivity & Progress Monitoring:** Provides visual dashboard stat cards (Total, Pending, Completed, High Priority) and an animated completion progress bar.
- **Interactive Search & Filtering:** Offers real-time title/description keyword search, tab filtering, and custom sorting (Newest, Priority, Title).

3. Advantages (Pros)

- [PRO] **Modern Full-Stack Integration:** Pairs a lightweight, robust Django 6 backend with a high-performance React 19 single-page frontend (SPA).
- [PRO] **Clean & Responsive User Interface:** Intuitive dashboard design featuring instant tab filtering, progress tracking, and custom CSS variables.
- [PRO] **Dark Mode & Light Mode Theme Toggle:** Built-in theme switcher with user theme preferences saved in localStorage.
- [PRO] **Instant Task Completion Toggle:** Allows users to mark tasks complete or pending directly from the task list without re-opening edit forms.
- [PRO] **CSRF Security Protection:** Native protection against Cross-Site Request Forgery via Django CSRF middleware and X-CSRFToken headers.
- [PRO] **Automated Test Suite:** 100% passing test coverage (``python manage.py test``) for user authentication, API endpoints, and template rendering.

4. Limitations (Cons) & Areas for Improvement

- [CON] **Basic Custom Authentication Model:** Current model uses custom ``User`` table; production apps should integrate Django's standard hashed ``AbstractUser`` / ``User`` authentication system.
- [CON] **Single-User Scope per Account:** Lacks multi-user team collaboration, shared workspace lists, or task assignment features.

- [CON] No File Attachments Support:** Tasks currently store text titles and descriptions, but do not support uploading document or image attachments.
- [CON] Lack of Push / Email Notifications:** Overdue tasks are visually flagged on the dashboard, but automated email reminders or browser push notifications are not yet implemented.
- [CON] SPA History Reload Handling:** Deep dynamic page reloads rely on server route handlers; adding a wildcard SPA fallback view will ensure 100% seamless direct URL bookmarks.

5. Summary Comparison Matrix

Category	Pros (Strengths)	Cons (Limitations)
Architecture	Django + React SPA, Fast Vite builds, REST APIs	Custom User model instead of Django's default auth
User Experience	Dark mode, live search, progress bar, tab filters	No push/email notifications for overdue tasks
Task Features	Priorities, due date status tags, quick completion toggle	No file attachments or sub-task checklists
Collaboration	Isolated per-user task dashboard and stats	No team sharing, comments, or multi-user assignment